Project 4: An Analysis of One Program, Three Ways

Jack Cannell, Luke Coverdale, Chris Ray

CIS 520 Group 14

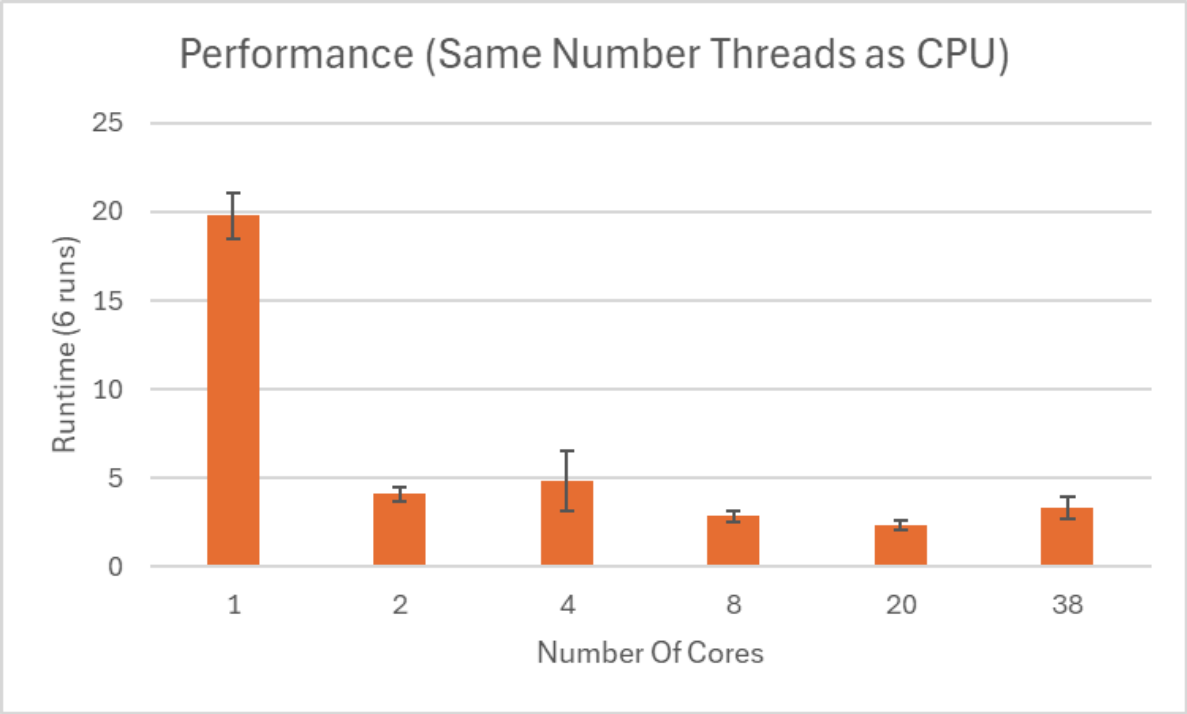
Kansas State University

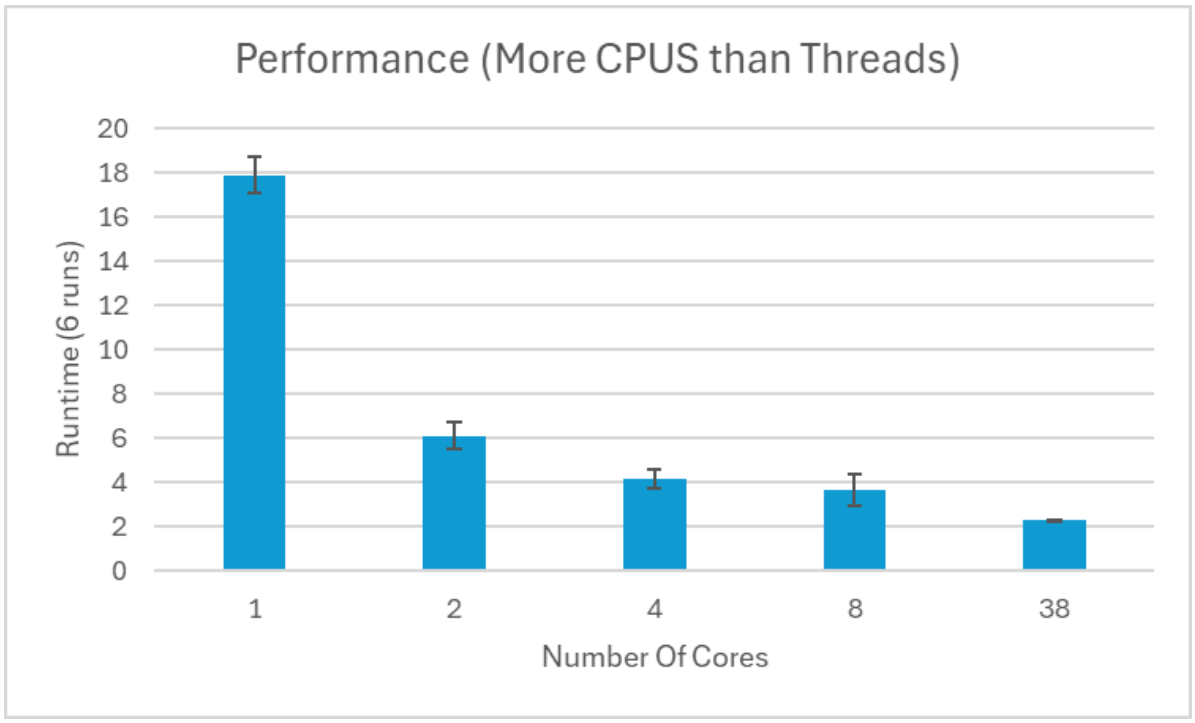
Introduction:

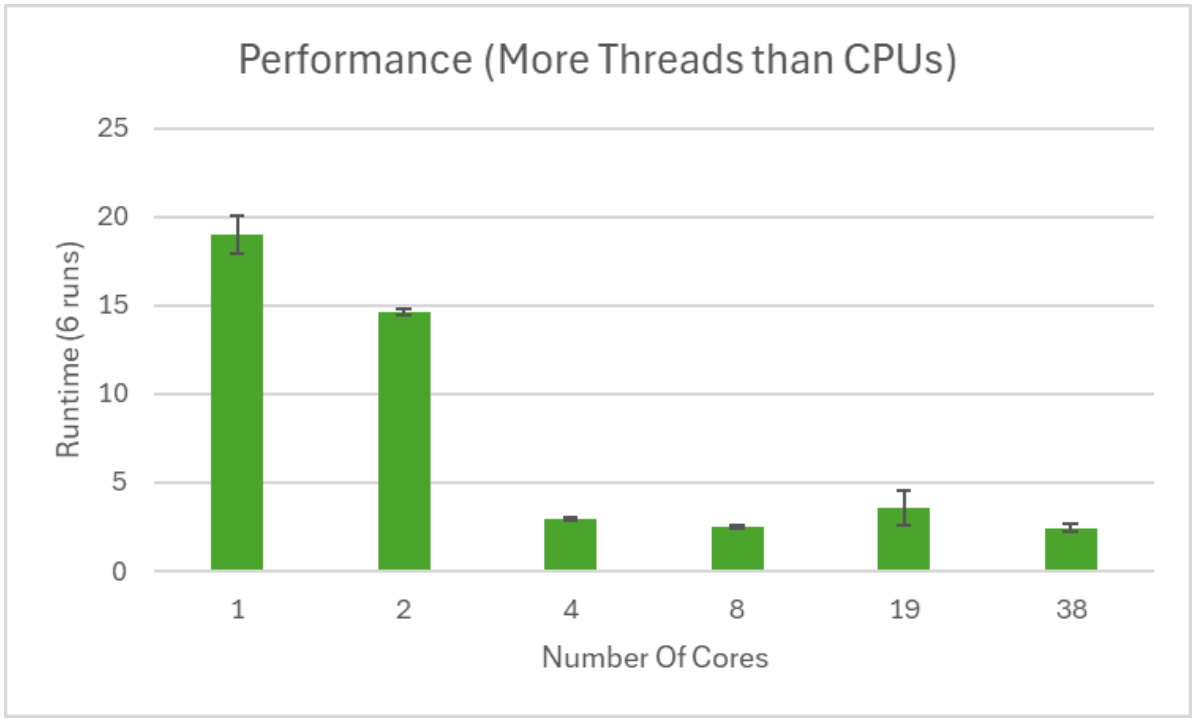
Since we have learned how virtual memory, multiple threads/programs, and system calls work, we can now analyze how parallel and distributed applications perform on a real operating system. We have been tasked with developing a parallelized program to find the maximum value of the ASCII characters in a line in the text file "wiki_dump.txt". We will do this with three standard tools for building parallelized programs: pthreads, MPI, and OpenMP. We will evaluate each implementation using multiple amounts of cores to show the performance differences across multiple machines vs. on a single machine. We will show the graphical results of these evaluations and will discuss the results in the subsequent sections.

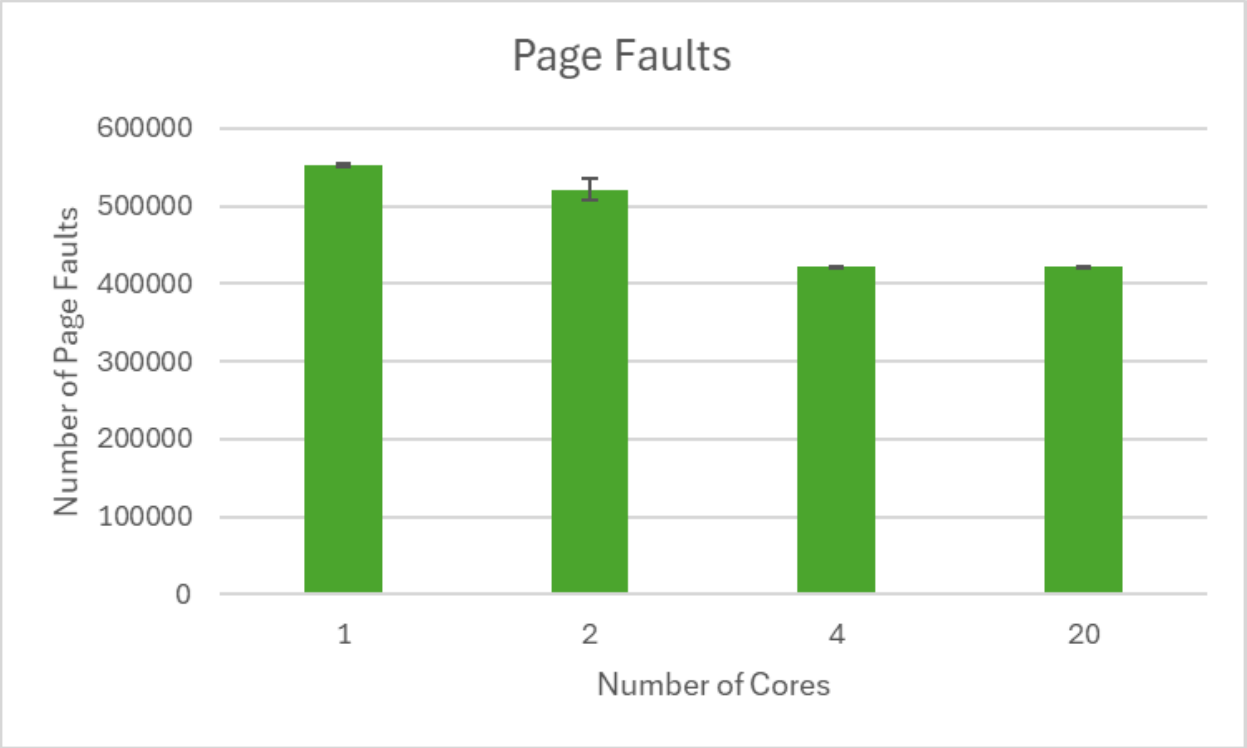
Pthreads Implementation:

The pthreads implementation takes in the number of CPUs it will have available from the job and then uses that to determine how many threads it should split the task into. It tries to split the task as evenly as possible between the threads.




Performance (More CPUS than Threads)
Runtime (6 runs)
20
18
16
14
12
10
8
6
4
2
0
1
2
4
8
38
Number Of Cores


Performance (More Threads than CPUs)
Runtime (6 runs)
25
20
15
10
5
0
1
2
4
8
19
38
Number Of Cores



Graph Analysis - Pthreads

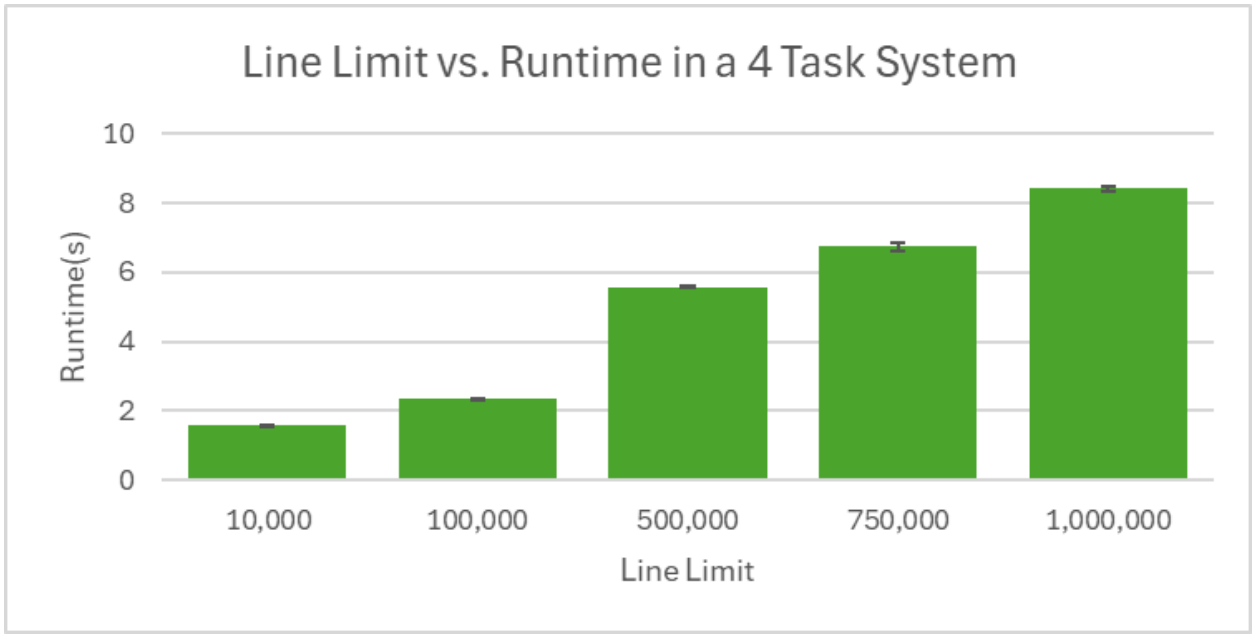
We noticed an odd occurrence while finding this data. Namely, adding more cores did not necessarily improve performance. After poking around looking at information on distributed systems, it seems that the time spent splitting up the task among different cores can overshadow the runtime due to oddities from Beocat. In our runs, we can see that there are nearly 100,000 more page faults for the example of more threads than cores. We believe this is the reason for the sudden and steep increase in performance between 2 and 4 cores and this similarly explains the absolutely massive increase in performance for 1 to 2 cores in our other graphs. This was a good reminder that throwing more cores or threads at a process does not necessarily mean better performance, there are other factors that affect the outcome.

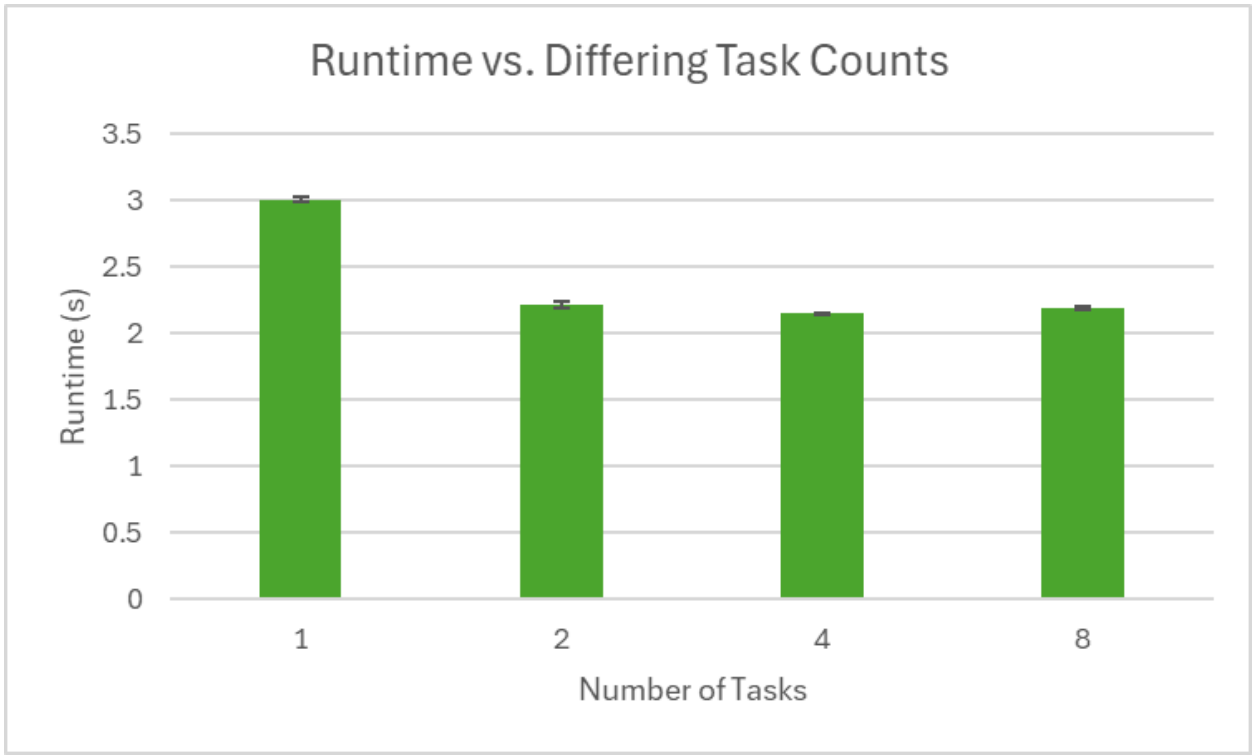
Pthreads memory

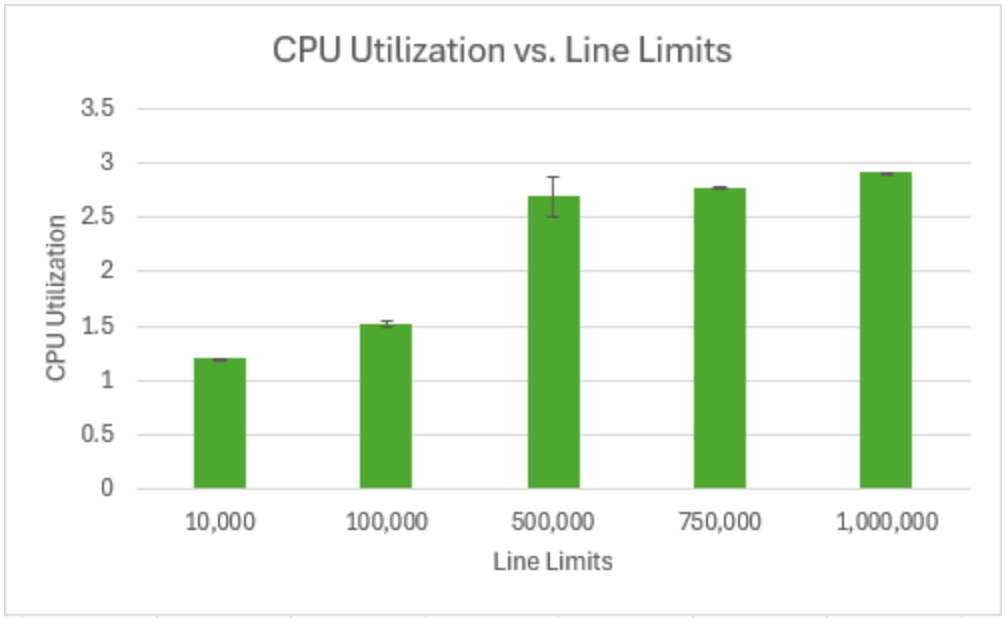
We used to have a graph showing memory for pthreads but that memory was erroneous. After using a different method, we found that the various configurations of pthreads makes extremely little difference on memory usage. This is because we load all the data into an array to start which dwarfs any difference in memory that might be from more cores or threads. The amount of memory used is about 1.7GB which is the size of the file.

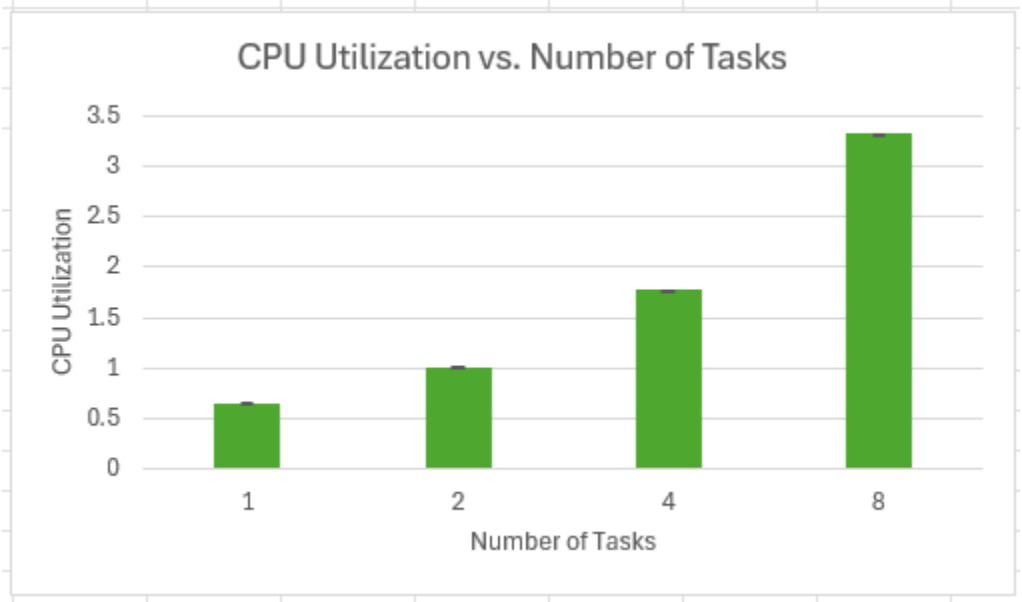
MPI Implementation:

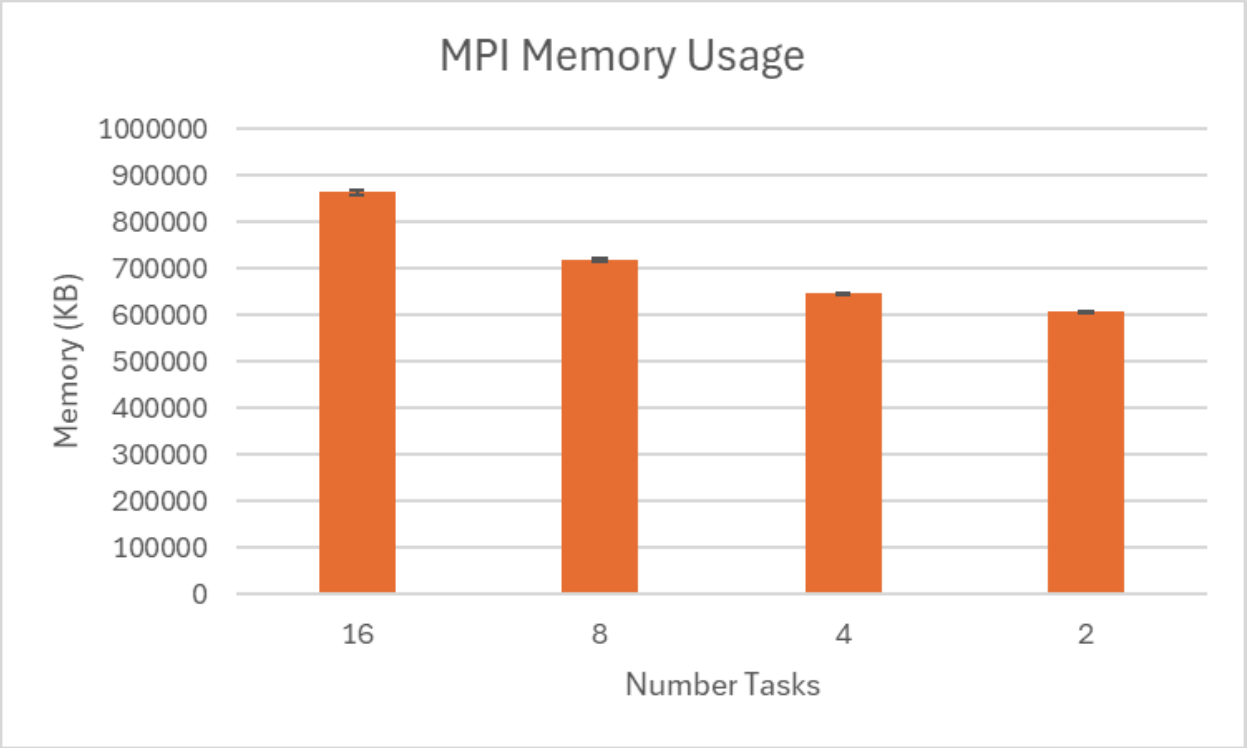
For this section, we used MPI to distribute our workload among multiple processes. First, we have to initialize at the beginning, and finalize the framework when we are done [Introduction to MPI - Research Computing Services]. Our master process is our process with rank 0, which handles many different tasks, including file reading, memory allocation, data preparation, and finally gathering and printing our final results. All of our worker processes (processes without rank 0) receive their assigned lines of data and compute their lines locally, sending their results back to the master process. We utilize MPI_Scatterv to dynamically split the workload based on our process count and number of lines. We then utilize MPI_Gatherv to collect the data once it has been computed locally. This local computation allows for good parallelization as each worker can work independently without communicating with another for their job. Our code does not include any race conditions because every MPI process works on its local copy of the data, and our master process gathers the data after all of our worker processes have finished their computations. The only real synchronization we do between processes is using MPI_Bcast to make sure all workers know the total number of lines. Our three main communications are the broadcast, the scatter, and the gather, with the scatter having the most overhead since it is sending lines of text to each worker. We optimize this communication through the use of Scatterv and Gatherv because they allow us to send uneven numbers of lines to each process [Using Collective Communication - Scatter vs. Scatterv]. We also flatten our data into a single character array before sending it through scatter so that we can avoid using a double pointer. This is as far as we optimized, because it was the simplest optimization to implement. Our initial implementation used MPI_Send and MPI_Recv, but we were running into memory issues with this implementation, so we switched to using scatter. We also ran into issues of truncation when having our MAX_LINE_LENGTH value be 1024 instead of 2048, as parts of the lines were not being read.


Line Limit vs. Runtime in a 4 Task System
Runtime(s)
10
8
6
4
2
0
10,000
100,000
500,000
750,000
1,000,000
Line Limit


Runtime vs. Differing Task Counts
Runtime (s)
3.5
3
2.5
2
1.5
1
0.5
0
1
2
4
8
Number of Tasks


CPU Utilization vs. Line Limits
CPU Utilization
3.5
3
2.5
2
1.5
1
0.5
0
10,000
100,000
500,000
750,000
1,000,000
Line Limits


CPU Utilization vs. Number of Tasks
CPU Utilization
3.5
3
2.5
2
1.5
1
0.5
0
1
2
4
8
Number of Tasks



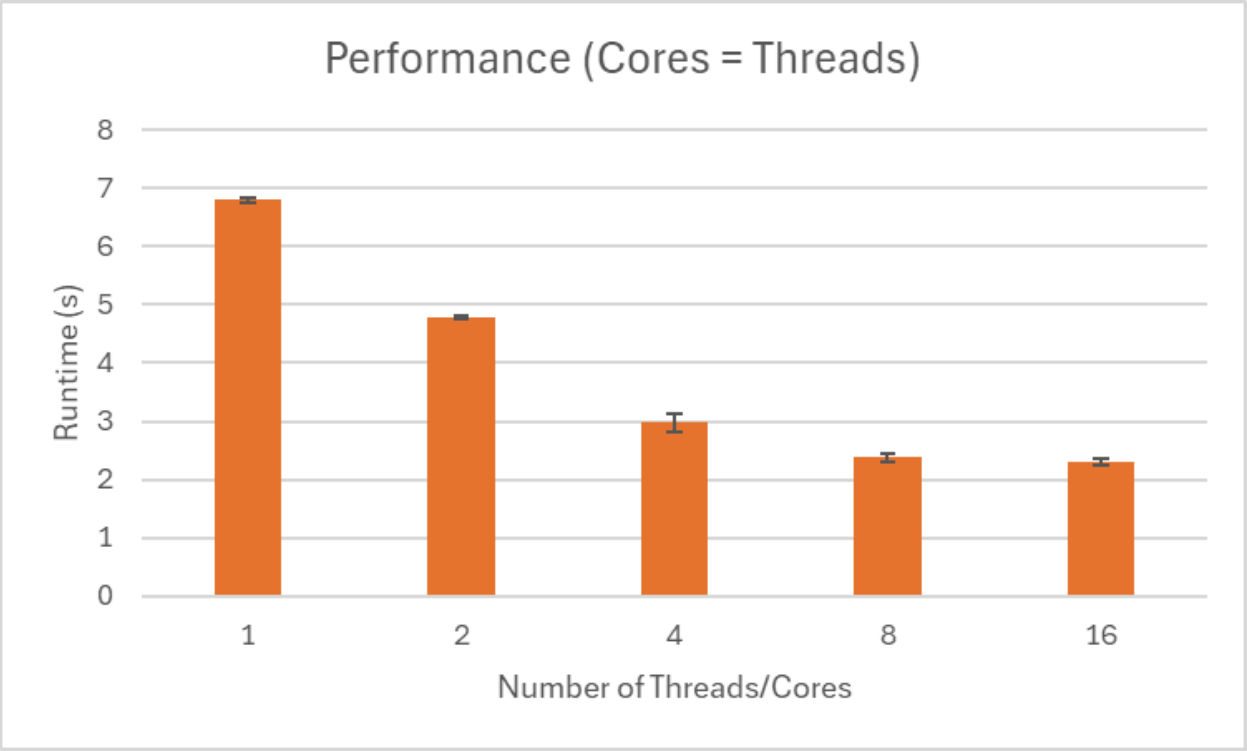
Graph Analysis - MPI

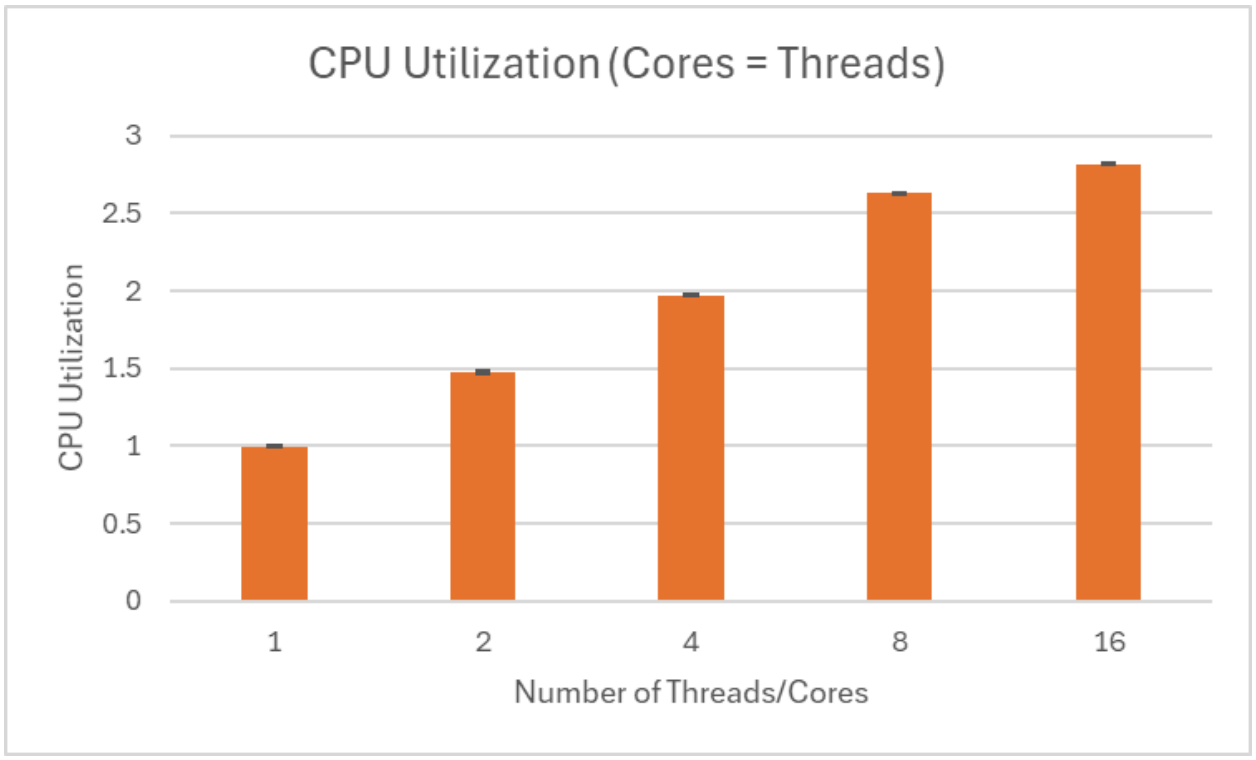
We can see that we get reasonable performance gains from increasing the number of tasks we have. This is expected but what needs an explanation is the slight bump when we go to 8 tasks. We believe this is because of the increased overhead of communication when increasing task numbers. This also explains why our CPU usage increased appropriately but we cannot see that in the wall clock time. That extra CPU time was spent communicating between the tasks. Lastly, the memory usage falls with the decrease in tasks as the number of variables assigned for each task is significantly larger than for pthreads or OpenMP.

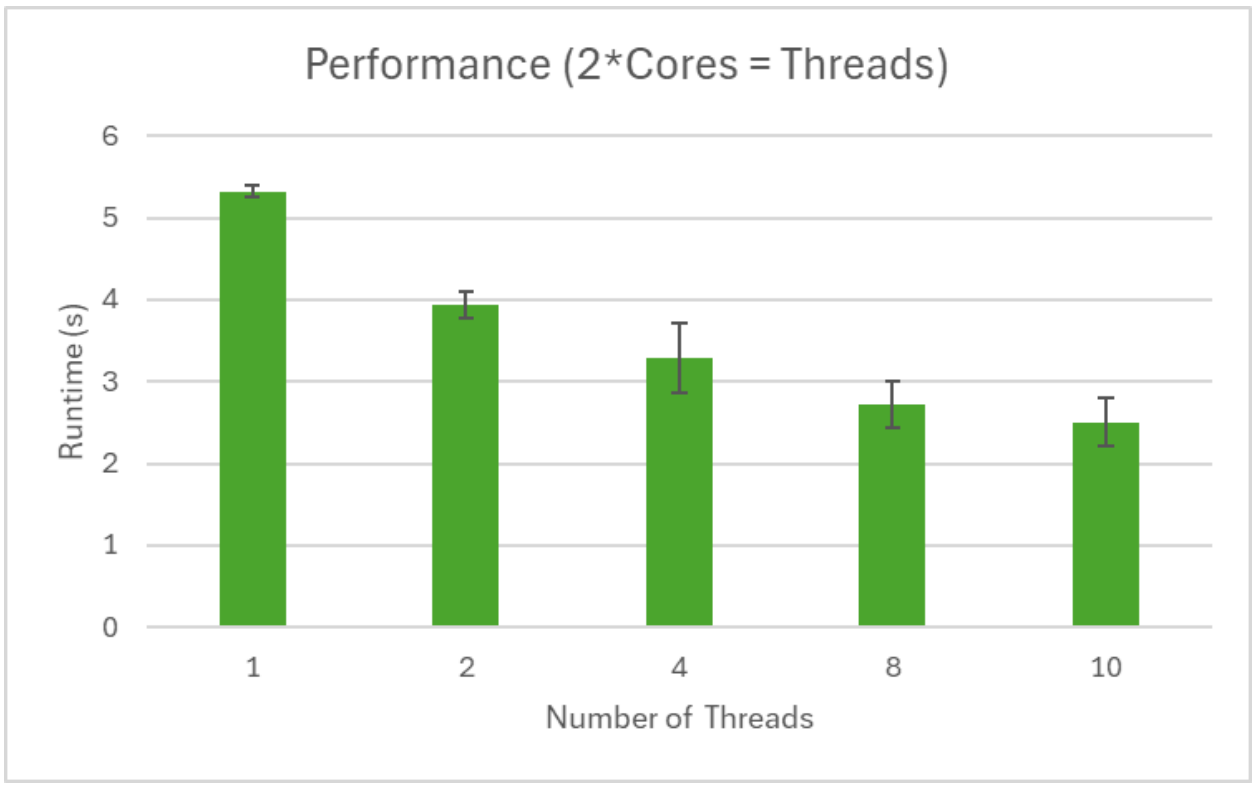
OpenMP Implementation:

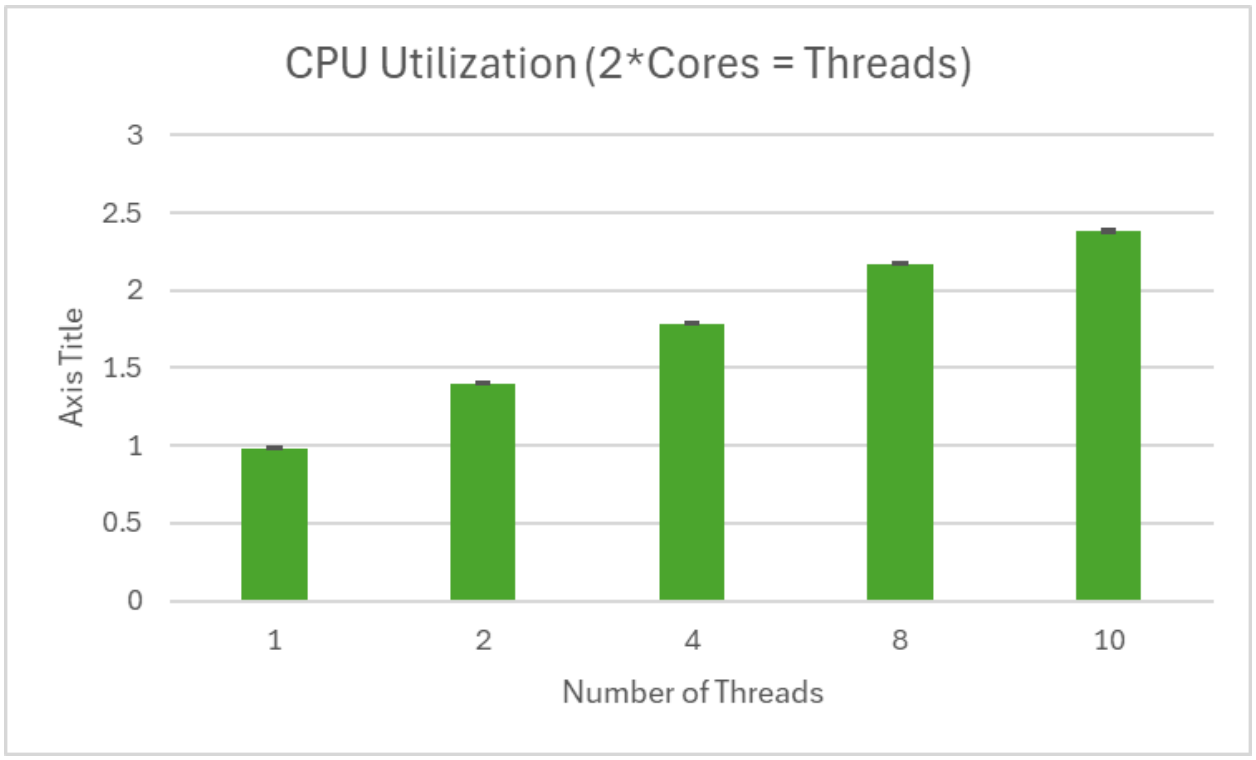
For this section, we used OpenMP to parallelize our program. Our OpenMP implementation was extremely similar to our pthreads implementation. Firstly, we open the file and check for errors when opening it. Then, we allocated the memory for the different variables (lines, max_values) based on the size of their variable type and the number of lines we were trying to read in. We make sure that this memory was allocated correctly, then we read in the lines from the file using getline(). We reallocate our memory if we are not large enough to read in the file, then we move to our OpenMP section. In this section, we set the number of threads using omp_get_max_threads() to calculate our location in our process_lines function [Using OpenMP

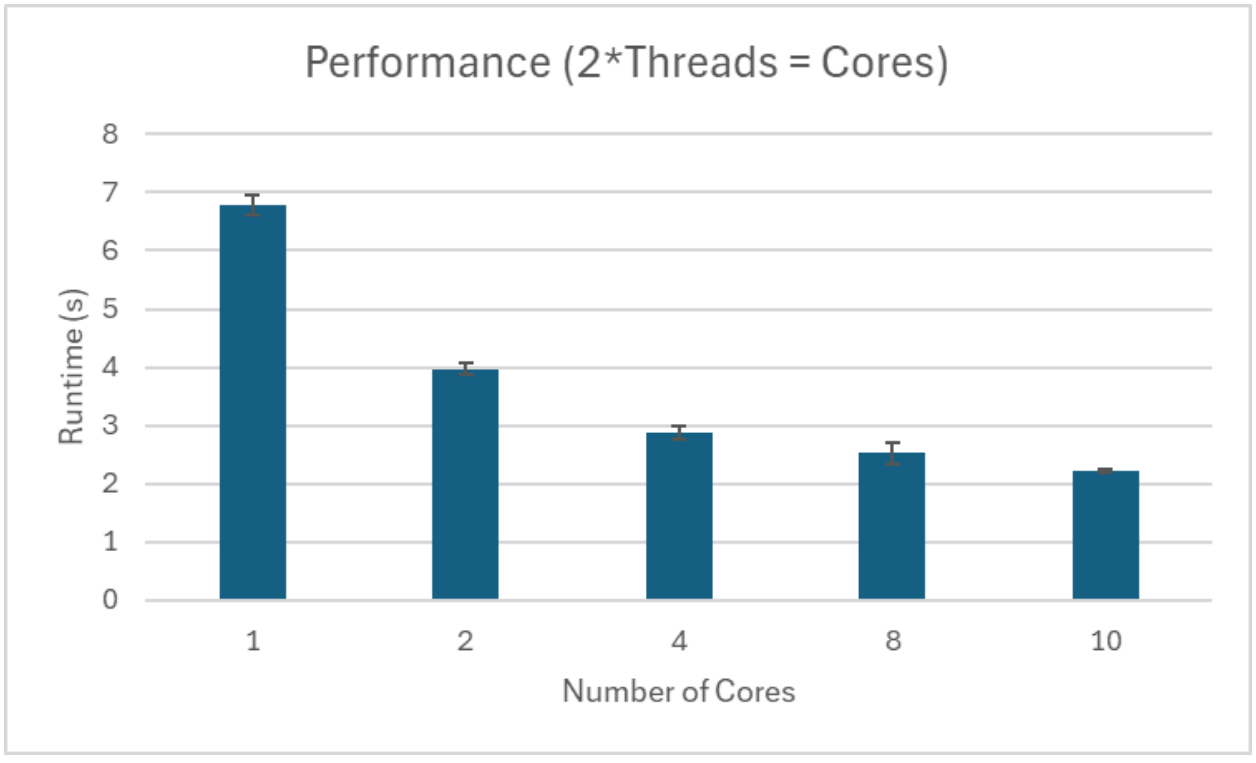
with C]. We then call that function, passing in our current thread number. We then print our maximum values array index by index, and free our memory. Our OpenMP implementation does not have any race conditions, because each thread works on an independent set of data. We do not have any synchronization between processes, because our implementation runs on one process. We also do not focus on communication, because our threads share a memory space, so they do not have to communicate with each other, and OpenMP handles each thread's execution automatically.

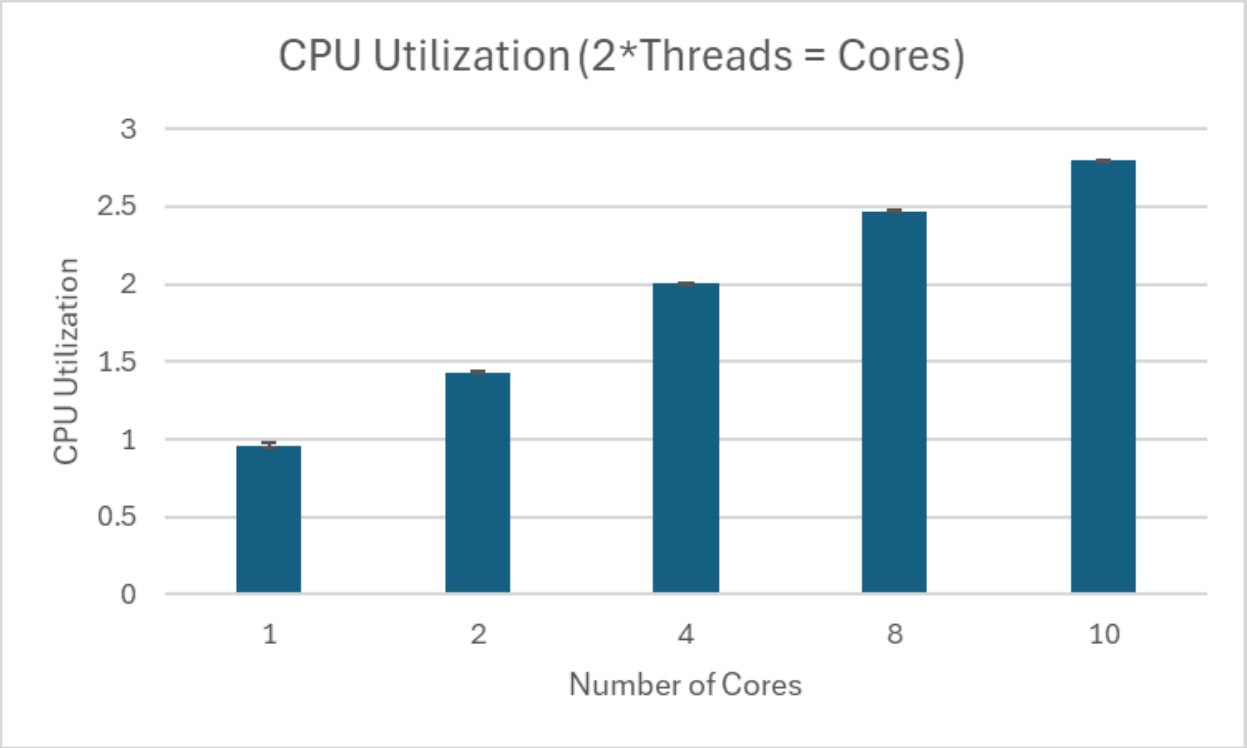



CPU Utilization (Cores = Threads)
CPU Utilization
3
2.5
2
1.5
1
0.5
0
1
2
4
8
16
Number of Threads/Cores


Performance (2*Cores = Threads)
Runtime (s)
6
5
4
3
2
1
0
1
2
4
8
10
Number of Threads


CPU Utilization (2*Cores = Threads)
3
2.5
2
1.5
1
0.5
0
Axis Title
1
2
4
8
10
Number of Threads


Performance (2*Threads = Cores)
8
7
6
5
4
3
2
1
0
Runtime (s)
1
2
4
8
10
Number of Cores



Graph Analysis -OpenMP

We can see for OpenMP, we get a much more normal change between threads and CPU numbers. Our performance increases nicely with each bump of threads when we have the same number of cores. We can also see that when we have twice as many threads as cores, the results are very similar to when we have twice the number of cores when cores = threads. To put another way, 1 core, 2 threads performs very similarly to 2 cores 2 threads. From this, we can assume each core allows for 2 threads and the slight lack of performance comes from communication issues between the threads. This hypothesis is further born out when we have twice as many cores as threads. These results are similar to when 1 core = 1 thread, thus showing that giving more cores does not increase performance when there is no application to use it.

Memory for OpenMP

There is no graph for memory usage for OpenMP because the difference in memory usage is essentially zero and is remarkably similar across all tests. This is because we load all lines into the program before delegating with OpenMP leading to the only difference being between the number of one private variable created per application. The amount of memory used is about 1.7GB which is the size of the file.

Appendices

Pthreads Shell Script:

Code Blame 9 lines (7 loc) · 340 Bytes

```
1  #!/bin/bash
2  #SBATCH --constraint=moles
3  #SBATCH --cpus-per-task=1
4  #SBATCH --mem-per-cpu=2G  # Increase memory per CPU to 2 because 1 is not enough
5  #SBATCH --time=01:00:00  # Set a time limit just incase something breaks
6
7  gcc pthread.c -o pthread.o -lpthread
8  perf stat -o perf_${SLURM_CPUS_PER_TASK}core.txt ./pthread.o $SLURM_CPUS_PER_TASK
```

Pthreads Output:

0: 125

1: 125

2: 125

3: 125

4: 125

5: 125

6: 125

7: 125

8: 125

9: 124

10: 226

11: 195

12: 125

13: 226

14: 195

15: 125

16: 125

17: 125

18: 125

19: 125
20: 125
21: 226
22: 125
23: 125
24: 125
25: 195
26: 125
27: 125
28: 226
29: 125
30: 125
31: 125
32: 125
33: 125
34: 214
35: 226
36: 226
37: 226
38: 125
39: 125
40: 125
41: 125
42: 125
43: 125
44: 226
45: 226
46: 195
47: 217
48: 125
49: 226

50: 226
51: 125
52: 195
53: 125
54: 125
55: 125
56: 226
57: 125
58: 125
59: 125
60: 125
61: 125
62: 125
63: 125
64: 226
65: 125
66: 125
67: 125
68: 226
69: 194
70: 194
71: 226
72: 125
73: 226
74: 197
75: 125
76: 226
77: 125
78: 224
79: 226
80: 209

81: 125
82: 195
83: 226
84: 125
85: 125
86: 226
87: 226
88: 125
89: 226
90: 226
91: 125
92: 206
93: 226
94: 195
95: 195
96: 125
97: 226
98: 226
99: 195
100: 125

MPI Shell Script:

Code Blame 13 lines (10 loc) · 468 Bytes

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=16 #Adjust based on number of processes we want (change for each test)
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=2G  # Increase memory per CPU to 2 because 1 is not enough
#SBATCH --time=01:00:00  # Set a time limit just incase something breaks
#SBATCH --constraint=moles


module load OpenMPI/4.1.4-GCC-11.3.0

mpicc -o MPI.o MPI.c
perf stat -r 10 -o perf_${SLURM_NTASKS}tasks.txt mpirun -np $SLURM_NTASKS ./MPI.o
```

MPI Output:

0: 125

1: 125

2: 125

3: 125

4: 125

5: 125

6: 125

7: 125

8: 125

9: 124

10: 226

11: 195

12: 125

13: 226

14: 195

15: 125

16: 125

17: 125

18: 125

19: 125

20: 125
21: 226
22: 125
23: 125
24: 125
25: 195
26: 125
27: 125
28: 226
29: 125
30: 125
31: 125
32: 125
33: 125
34: 214
35: 226
36: 226
37: 226
38: 125
39: 125
40: 125
41: 125
42: 125
43: 125
44: 226
45: 226
46: 195
47: 217
48: 125
49: 226
50: 226

51: 125
52: 195
53: 125
54: 125
55: 125
56: 226
57: 125
58: 125
59: 125
60: 125
61: 125
62: 125
63: 125
64: 226
65: 125
66: 125
67: 125
68: 226
69: 194
70: 194
71: 226
72: 125
73: 226
74: 197
75: 125
76: 226
77: 125
78: 224
79: 226
80: 209
81: 125

82: 195
83: 226
84: 125
85: 125
86: 226
87: 226
88: 125
89: 226
90: 226
91: 125
92: 206
93: 226
94: 195
95: 195
96: 125
97: 226
98: 226
99: 195
100: 125

OpenMP Script:

Code Blame 7 lines (7 loc) · 179 Bytes

```
1  #!/bin/bash
2  #SBATCH --constraint=moles
3  #SBATCH --nodes=1
4  #SBATCH --mem-per-cpu=2G
5  export OMP_NUM_THREADS=4
6  gcc -fopenmp OpenMP.c -o OpenMP.o
7  perf stat -o perf_1core.txt ./OpenMP.o
```

OpenMP Output:

0: 125

1: 125

2: 125

3: 125

4: 125

5: 125

6: 125

7: 125

8: 125

9: 124

10: 226

11: 195

12: 125

13: 226

14: 195

15: 125

16: 125

17: 125

18: 125

19: 125

20: 125
21: 226
22: 125
23: 125
24: 125
25: 195
26: 125
27: 125
28: 226
29: 125
30: 125
31: 125
32: 125
33: 125
34: 214
35: 226
36: 226
37: 226
38: 125
39: 125
40: 125
41: 125
42: 125
43: 125
44: 226
45: 226
46: 195
47: 217
48: 125
49: 226
50: 226

51: 125
52: 195
53: 125
54: 125
55: 125
56: 226
57: 125
58: 125
59: 125
60: 125
61: 125
62: 125
63: 125
64: 226
65: 125
66: 125
67: 125
68: 226
69: 194
70: 194
71: 226
72: 125
73: 226
74: 197
75: 125
76: 226
77: 125
78: 224
79: 226
80: 209
81: 125

82: 195
83: 226
84: 125
85: 125
86: 226
87: 226
88: 125
89: 226
90: 226
91: 125
92: 206
93: 226
94: 195
95: 195
96: 125
97: 226
98: 226
99: 195
100: 125